

TellerX

系统处理流程说明

问题问答链路 · 原始文档入库链路 · 状态机 · 数据落点

当前代码基线	codex/postgresql-pgvector-fts
整理日期	2026-08-26
文档目的	帮助研发、测试和运维人员理解端到端调用链与持久化边界

核心原则：先保存不可丢失的事实，再异步生成可重建的搜索投影；先取得证据，再生成回答；先校验引用，再返回结论。

1. 架构边界

系统把存储和处理职责分成五个连续层次。左侧是不可丢失的输入，右侧是面向问答的派生能力。

原始文件对象 内容寻址、不可覆盖	PostgreSQL 事实表 Document / Version / Chunk	Outbox + 向量对象 可靠发布、可复用派生产物	搜索投影 FTS + pg_trgm + pgvector	证据问答 检索、生成、引用校验
---------------------	---	-------------------------------	-------------------------------------	--------------------

权威事实与派生投影

- 原始文件、DocumentVersion 和 Chunk 是事实来源。
- 规范化 JSON 和向量对象是可审计、可复用的中间产物。
- chunk_search_index 是为了检索性能生成的投影，可以通过 Outbox 和 reconcile 重建。
- 上传接口返回 202 只代表文件已接收并创建任务，不代表已经可以检索。
- 问答接口只返回通过服务器端引用检查的声明。

两条核心链路

链路	起点	终点	关键安全门
问题问答	用户问题	回答 + claims + sources	主题落地、引用 ID、原文 quote、实时版本
文档入库	原始文件	searchable/current 搜索投影	SHA 去重、任务租约、Outbox、数量与清单校验

2. 用户问题处理流程

主调用链从 React 表单开始，经过 FastAPI、查询理解、混合检索、模型路由和引用校验，最后持久化并返回。

01	前端提交 Composer -> App.submit() -> askKnowledgeBase()	question / conversation_id / project_ids
02	API 契约 ChatRequest 校验问题长度和字段类型	POST /api/v1/chat
03	项目范围 未传项目且项目多于1个时返回 422	确定检索边界
04	会话与追踪 AnswerService 创建 trace_id，获取或创建 Conversation	开始计时
05	查询理解 规则计划、缓存或 Plus 模型生成 QueryPlan	subjects / identifiers / constraints
06	混合检索 原问题基线通道 + QueryPlan 规划通道	Evidence[]
07	无证据分支 不调用生成模型，直接生成确定性拒答	insufficient_evidence
08	模型路由 冲突、跨文档或复杂问题走 Max，其余走 Plus	requested_tier
09	证据 Prompt 按 4000/7000 Token 预算序列化来源	QUESTION + REQUESTED_FACTS + EVIDENCE
10	结构化生成 Qwen JSON Object，temperature=0	status / claims / evidence quotes
11	引用校验 验证状态、claim、Evidence ID 和连续原文 quote	ValidatedAnswer
12	实时来源校验 落库前确认引用仍为 searchable/current	阻止版本切换竞态
13	持久化与返回 写 Message + QueryTrace，返回 ChatResponse	前端显示回答和原文

3. 查询理解与混合检索

查询理解

01	规则基线 fallback_query_plan 提取主题和精确 ID	永远可用
02	简单查找 只有一个实体名时直接返回规则计划	不调用模型
03	缓存检查 按规范化问题使用 LRU + TTL 缓存	默认 500 条 / 3600 秒
04	语义计划 Plus 模型只做检索规划，不回答问题	最多4条 retrieval_queries
05	计划清洗 重新分类主题、场景、请求字段和约束	QueryPlan
06	安全降级 模型、JSON 或空计划失败时回到规则计划	不中断问答

检索参数

参数	默认值	作用
retrieval_top_k	50	每个初始召回通道的候选上限
rerank_candidates	30	送入重排器的候选上限
evidence_top_k	8	核心证据数量
vector_min_similarity	0.25	向量最低相似度

4. 混合检索内部流程

01	双通道 先执行原问题基线检索，再执行 QueryPlan 规划检索	基线证据优先
02	词法召回 PostgreSQL FTS + 精确 ID + 业务信号 + pg_trgm	SearchIndex.lexical_search
03	向量召回 查询向量缓存或 Qwen embeddings，再用 pgvector HNSW	SearchIndex.vector_search
04	RRF 融合 按名次融合不同分值尺度，并轻微提升 approved	统一候选顺序
05	主题落地 语义主题未出现在任何候选时主动返回空证据	防止错误主题扩散
06	跨文档扩展 从已命中主题发现 ID/别名，并逐个做精确词法检索	确定性关系扩展
07	候选补充 结果少于3条时补充 approved + draft	approved 保持优先
08	规则过滤 内容去重、精确 ID 全覆盖、完整实体匹配	减少误召回
09	相邻分块 取相关文档有序 Chunk，保留标题后的正文值块	补足上下文
10	模型重排 Qwen rerank 后补精确信号并做文档多样化	默认核心证据8条
11	来源桥 补充主题到下游 ID 的映射证据，不创造新事实	Evidence[]

检索范围统一要求：文档未删除、版本 technical_status=searchable、approved 必须是 current、符合 project 范围。当前聊天入口未传 principal_ids，因此只检索 public 文档。

5. 模型生成与最终结论

路由与故障转移

- 冲突、跨多个文档或包含比较/差异/综合等标记时选择 Max，否则选择 Plus。
- 本地成功 Token 用量达到 80% 告警，达到 90% 停止派发。
- 未固定模型时按同层优先级故障转移；固定模型不跨模型降级。
- Max 完全不可用时，非固定请求允许一次受证据约束的 Plus 降级。

引用校验门

01	状态校验 只允许 answered / conflict / insufficient_evidence	非法即拒绝
02	Claim 校验 answered/conflict 必须至少有一条事实声明	每条非空
03	引用存在 每条 claim 必须携带一个或多个 evidence 引用	不可无来源
04	ID 边界 引用 chunk_id 必须属于本次 Evidence	禁止模型自造来源
05	原文校验 quote 必须是来源中的连续原文	仅有限修复省略号/排版
06	冲突校验 conflict 至少引用两个证据 ID	不能单证据宣称冲突
07	重新拼接 最终 answer 由已校验 claim.text 按行拼接	不直返模型顶层 answer
08	失败重试 追加引用纠正提示并重新生成一次	仍失败则确定性拒答

最终实现：answer = "\n".join(claim.text for claim in claims)

6. 原始文档入库主流程

入库链路分为上传受理、事实处理和搜索投影发布三个阶段。

01	请求校验 检查生命周期、文件扩展名和表单字段	HTTP 4xx 或继续
02	原始对象 按1MB流式读取，限制大小并计算 SHA-256	内容寻址不可覆盖
03	项目身份 查询或并发安全地创建 Project	项目名唯一
04	文档身份 按 project_id + logical_key 查询或创建 Document	稳定逻辑身份
05	版本去重 按 document_id + sha256 判断重复	复用版本或创建新版本
06	任务创建 DocumentVersion=received，IngestionJob=queued	提交后返回 202
07	任务领取 SKIP LOCKED + 15分钟租约	多 Worker 安全并发
08	格式解析 DOCX/PDF/HTML/Excel/CSV 等转为 ParsedUnit	保留来源位置
09	文本切块 按结构、段落、句子和最大650 Token 生成 TextChunk	非表格添加重叠
10	规范化产物 保存 JSON.zlib 并写 DocumentArtifact	可审计中间结果
11	向量处理 缓存命中则复用，缺失内容每10条一批生成	对象存储 + 元数据
12	事实落库 写 Chunk、ChunkEmbedding 和相邻关系	稳定 UUID5
13	事务 Outbox 同一事务创建 OutboxEvent	Version/Job=index_pending
14	索引发布 Index Worker UPSERT chunk_search_index	FTS + pg_trgm + pgvector
15	校验切换 数量校验后切换 approved current 版本	Job=complete 100%

7. 格式解析与切块

格式解析矩阵

格式	解析行为	来源位置
DOCX	按段落/表格原始顺序；识别 Heading；表格单独成单元	标题路径、表格编号
PDF	每页一个逻辑单元；修复视觉换行拆开的 ASCII ID；无 OCR	页码、Page N
Markdown	按 # 标题层级切分正文	完整标题路径
HTML	删除脚本/样式；保留标题、段落、列表、引用和表格	标题路径
XLSX/XLSM	忽略隐藏 Sheet；公式 + 缓存值；每25行一组	Sheet、单元格范围
CSV	每25行一组，后续组重复表头	行范围
DOC/XLS	LibreOffice 临时转换后复用现代解析器	转换 warning
TXT	整份文本形成一个初始单元	无额外定位

切块规则

01	自然边界 ParsedUnit 已保留标题、页码和表格结构	先按空行拆段
02	长段落 超过 max_tokens=650 时按中英文句末标点拆句	尽量不破坏句义
03	超长单句 仍超过上限时退化为字符窗口	保证硬上限
04	正文重叠 下一块前置上一段末尾60个空白分隔项	补上下文
05	表格保护 表格不机械重叠，避免坐标和内容重复	引用位置准确
06	稳定元数据 生成 ordinal/content_hash/token_count 和来源定位	TextChunk

当前代码会删除 target_tokens 参数，因此实际切块主要受解析器自然结构和 max_tokens 控制；target_tokens 目前只进入 chunker_fingerprint。

8. 向量缓存、Chunk 与 Outbox

向量缓存

01	向量空间 模型 ID + 维度 + 预处理版本生成 embedding_fingerprint	EmbeddingModel
02	内容去重 按 content_hash + fingerprint 查询缓存	EmbeddingCache
03	批量生成 缺失内容每10个 Chunk 一批调用 embeddings	校验数量和维度
04	对象存储 float32 小端序列化并 zlib 压缩	embeddings/fingerprint/hash.f32.zlib
05	缓存元数据 保存 object_uri、checksum 和 dimensions	向量本体不放普通列
06	Chunk 关联 ChunkEmbedding 指向缓存，可跨文档复用同一正文向量	减少重复调用
07	降级 允许时缺失向量仍继续词法入库，否则任务失败	bm25_only 或 failed_final

Chunk 与事务 Outbox

- 重试时替换同一版本旧的 Chunk 和 ChunkEmbedding。
- Chunk ID 使用固定 namespace 的 UUID5：version_id + ordinal + content_hash。
- record_hash 同时包含正文和来源位置，用于搜索投影漂移校验。
- 先 flush 父 Chunk，再写 ChunkEmbedding，保证外键顺序。
- Chunk、向量关联、OutboxEvent 和 index_pending 状态在同一 ORM 事务提交。

9. 搜索投影发布与版本切换

01	领取事件 Index Worker 使用 SKIP LOCKED 和5分钟租约	pending -> processing
02	加载事实 读取 DocumentVersion、Document、Project 和有序 Chunk	版本快照
03	加载向量 从对象存储读取并校验 checksum/dimensions	当前 fingerprint
04	构造文本 raw_text；文件名 Token x4；标题 Token x3；正文 Token x1	exact_terms + lexical_text
05	UPSERT 写 chunk_search_index 的 FTS、trigram 和 vector 字段	独立数据库事务
06	清理残留 删除该版本不再存在的陈旧投影	版本快照一致
07	数量校验 实际投影数必须等于预期 Chunk 数	不完整则拒绝切换
08	同步记录 保存 IndexSyncState 和 manifest_hash	verified
09	旧版退役 旧 current -> deprecated/superseded/effective_to	先释放唯一位置
10	新版生效 新 approved -> searchable/current	indexed_at/searchable_at
11	任务完成 OutboxEvent=published，Job=complete 100%	可供问答检索
12	自动修复 reconcile 比较数量和 record_hash 清单	发现漂移则重建

发布当下直接校验投影数量；定时 reconcile 进一步比较 ordinal + chunk_id + record_hash 清单。Outbox 重试和 reconcile 共同保证异常中断后的最终收敛。

10. 状态机

DocumentVersion 技术状态

起始状态	目标状态	触发条件
received	parsing	入库 Worker 开始处理
parsing	chunked	解析、切块和规范化产物完成
chunked	embedded	所有唯一正文都有向量
chunked	bm25_only	向量不可用且允许降级
embedded / bm25_only	index_pending	Chunk 和 Outbox 已提交
index_pending	searchable	搜索投影发布并验证
index_pending	deleted	deprecated 删除事件发布
searchable	superseded	新批准版本取代
received/parsing/chunked	failed_final	入库处理失败

IngestionJob 与 OutboxEvent

对象	正常路径	失败/恢复路径
IngestionJob	queued -> running -> index_pending -> succeeded	running -> failed ; index_pending -> retry/dead
OutboxEvent	pending -> processing -> published	processing -> pending ; 第5次失败 -> dead ; 租约过期 -> pending

11. 最终数据位置

数据	位置	性质	可重建
原始文件	storage_root/{sha...}-{filename}	权威对象	否
文档身份	documents	权威事实	否
文件版本	document_versions	权威事实	否
入库任务	ingestion_jobs	工作流审计	否
规范化单元	JSON.zlib + document_artifacts	中间产物	从原文件
文本分块	chunks	权威事实	从解析产物
向量空间	embedding_models	元数据	是
压缩向量	embeddings/*.f32.zlib	派生产物	是
向量缓存	embedding_cache	派生元数据	是
Chunk 向量关联	chunk_embeddings	派生关系	是
发布事件	outbox_events	可靠工作流	不应删除
搜索投影	chunk_search_index	派生投影	是
同步校验	index_sync_state	一致性审计	是
问答消息	messages	问答记录	否
查询追踪	query_traces	问答审计	否

结论：chunk_search_index 不是文档内容的唯一存储。即使投影损坏，也可以从 DocumentVersion、Chunk 和向量对象重新发布。

12. 核心文件索引

文件	主要职责
frontend/src/App.jsx	问题提交、回答状态和本地会话快照
frontend/src/api.js	浏览器 HTTP 适配器
app/api/routes/chat.py	问答入口和项目范围检查
app/services/answering.py	问答总编排、实时引用检查和持久化
app/services/query_understanding.py	语义查询计划和规则降级
app/services/retrieval.py	混合召回、扩展、重排和证据整形
app/services/answer_contract.py	Prompt、证据预算和引用校验
app/services/model_router.py	Plus/Max 路由、配额和故障转移
app/api/routes/documents.py	文档上传、版本、审批、废弃和下载
app/integrations/storage.py	原始文件、解析产物和向量对象存储
app/knowledge/parsers.py	格式感知的文档解析
app/knowledge/chunking.py	文本和表格切块
app/services/ingestion.py	入库任务、向量缓存、Chunk 和 Outbox
app/jobs/ingestion_worker.py	入库 Worker 入口
app/services/indexing.py	搜索投影发布、验证、版本切换和修复
app/jobs/index_worker.py	Outbox/索引 Worker 入口
app/integrations/search.py	PostgreSQL FTS、pg_trgm 和 pgvector
app/db/models.py	ORM 事实表、工作流和审计模型
alembic/versions/0004_postgresql_pgvector_fts.py	搜索投影表及数据库索引迁移

配套可编辑版本：[docs/system-processing-flows.md](#)